Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Реализация множеств на основе битовых полей»

Выполнил:

студент группы 3825Б1ФИсп1

Марков А.А.

Проверил:

Ивлев А.Д.

Нижний Новгород

2026

Содержание

Введение 3

1. Постановка задачи 4

2. Руководство пользователя 5

3. Руководство программиста 6

3.1. Описание структуры программы 6

3.2. Описание структур данных 6

3.3. Описание алгоритмов 7

4. Результаты экспериментов и тестирования 9

Заключение 11

Литература 12

Введение

Для представления множеств над конечным множеством универса удобно использовать битовые поля. Каждому элементу в битовом поле сопоставляется один бит, отражающий наличие/отсутствие элемента в поле. Такой способ хранения обеспечивает компактность по памяти и позволяет реализовывать операции над множествами через побитовые операции.

В данной лабораторной работе рассматривается реализация множеств на основе битовых полей на языке C++. Разработаны классы TbitField и TSet для работы с битовыми полями и множествами.

Целью работы является изучение представления множеств, освоение приёмов работы с битовыми операциями, а также написание тестов с помощью GoogleTest. Корректность реализации проверяется с помощью набора тестов, охватывающих все функции и граничные случаи в них.

1. Постановка задачи

Требуется реализовать на языке C++ два класса.

1) Класс TbitField:
   – Создание битового поля заданной длины
   – Конструктор копирования и оператор присваивания
   – Получение длины битового поля
   – Установка, сброс и получение значения отдельного бита
   – Сравнение битовых полей на равенство
   – Побитовые операции
   – Операции потокового ввода/вывода
2) Класс TSet:
   – Создание множества на основе заданной максимальной мощности, а также на основе существующего битового поля
   – Конструктор копирования и оператор присваивания
   – Получение максимальной мощности множества
   – Проверку принадлежности, добавление и удаление элемента
   – Сравнение множеств на равенство
   – Добавление и удаление элемента в множестве с использованием int
   – Объединение и пересечение множеств одного универса
   – Операция дополнения множества
   – Операции потокового ввода/вывода

Также требуется разработать набор тестов (GTest) покрывающих все операции над классами с учётом граничных случаев.

2. Руководство пользователя

Разработанная программа представляет собой набор тестов для классов TBitField и Tset, а также .h файлы с реализацией классов. Взаимодействие осуществляется только через запуск файла с тестами и просмотр результатов тестирования.

Для сборки программы необходимо использовать среду разработки Microsoft Visual Studio с установленной поддержкой GTest. Запуск тестов осуществляетя двумя способами:

– В среде разработки сочетанием клавиш Ctrl+F5 (Запуск без отладки)
– Путём вызова исполняемого файла GTest.exe.

Вывод программы имеет следующий вид:

```
[==========] Running 18 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 9 tests from TestTBitField
[ RUN      ] TestTBitField.Create_BitField
[       OK ] TestTBitField.Create_BitField (0 ms)
...
[  PASSED  ] 18 tests.
```

Если тест не прошёл, то выводится `[ FAILED ]` вместе с информацией о том, какой тест конкретно не выполнился.

Дополнительно можно использовать классы в других программах как библиотеки. Для этого нужно подключить заголовочные файлы tbitfield.h и tset.h, и добавить соответствующие файлы реализации в проект.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей:

– tbitfield.h объявление класса TBitField

– tset.h объявление класса TSet

– tbitfield_.h реализация методов класса TBitField

– tset_.h реализация методов класса TSet

– test.cpp набор тестов GTest

3.2. Описание структур данных

Используется typedef unsigned int TELEM, а также константа

const int bits_in_telem = sizeof(TELEM) * 8

Класс TBitField:

Поля:

– int BitLen // длина битового поля

– TELEM *pMem // память для предоставления битового поля

– int MemLen // количество элементов Mem для представления битового поля

Приватные методы:

```
int TBitField::GetMemIndex(const int n) const {
   return n / bits_in_telem;
}

TELEM TBitField::GetMemMask(const int n) const {
   return TELEM(1) << (n % bits_in_telem);
}
```

Класс TSet:

Поля:

– int MaxPower; // максимальная мощность множества
– TBitField BitField; // битовое поле для хранения характеристического вектора

3.3. Описание алгоритмов

Битовое поле

Конструктор копирования (TBitField): скопировать поля BitLen и MemLen, выделить тот же массив pMem новой длины, поэтапно скопировать содержимое массива.

Установка бита (SetBit): проверить границы, вычислить индекс элемента idx, вычислить маску mask, выполнить побитовое ИЛИ pMem[idx] |= mask.

Очистка бита (ClrBit): проверить границы, вычислить индекс элемента idx, вычислить маску mask, выполнить побитовое И с инвертированной маской pMem[idx] &= ~mask.

Чтение бита (GetBit): проверить границы, вычислить индекс элемента idx, вычислить маску mask, вернуть (pMem[GetMemIndex(n)] & GetMemMask(n)) != 0

Операторы сравнения (operator== и operator!=): сравнить длины полей, почленно сравнить элементы, вернуть результат сравнения.

Оператор присваивания (operator=): проверить совпадает ли размер выделенной памяти текущего объекта и объекта-источника, если размеры не совпадают то выделить новую память нужного размера, поэлементно скопировать данные из одного массива в другой, вернуть ссылку на текущий объект.

Операция ИЛИ (operator |): взять максимум длин двух полей, создать новое поле этой длины, для каждого элемента i выполнить побитовое суммирование |

Операция И (operator &): взять максимум длин двух полей, создать новое поле этой длины, для каждого элемента i выполнить побитовое умножение &

Операция инверсия (operator ~): создать поле под результат такой же длины, для каждого элемента выполнить побитовое отрицание ~, в последнем элементе приравнять «лишние» биты к 0

Ввод (operator>>): очистить все биты, посимвольно считывать символы 0 и 1, пока не закончится поток или не будет достигнута длина поля

Вывод (operator<<): выводить в цикле (от 0 до BitLen-1) 0 или 1 в зависимости от значения бита

Множество

Добавление элемента (InsElem): вызвать SetBit для элемента

Удаление элемента (DelElem): вызвать ClrBit для элемента

Проверка принадлежности (IsMember): вызвать GetBit для элемента

Сравнение множеств (operator== и operator!=): сравнить по мощности и битовым полям, вернуть результат сравнения

Оператор присваивания (operator=): проверить на самоприсваивание, скопировать мощность множества и битовое поле, вернуть текущий объект

Объединение/разность с элементом (operator+ и operator-): создать временную копию текущего множества, добавить/удалить заданный элемент, вернуть полученное множество

Объединение множеств (operator+): проверить равенство мощностей, выпонить побитовое ИЛИ над битовыми полями множеств, вернуть новый объект

Пересечение множеств (operator*): проверить равенство мощностей, выполнить побитовое И над битовыми полями множеств, вернуть новый объект

Дополнение множества (operator~): применить инверсию к битовому полю множества, вернуть новый объект

Ввод (operator>>): очистить все биты множества, считывать целые числа из потока пока не закончится ввод, если введённое число находится в диапазоне размера множества то добавить его

Вывод (operator<<): пройти циклом по числам от 0 до MaxPower-1, если число принадлежит множеству вывести его

4. Результаты экспериментов и тестирования

Для проверки корректности работы классов было проведено тестирование с использованием библиотеки GoogleTest. Тестирование проводилось в среде разработки Microsoft Visual Studio. В ходе эксперимента запущено 18 тестов, разделённых на два набора.

Набор проверок TestTBitField (9 тестов):

1) Create_BitField: создание битового поля корректной длины с проверкой на исключение при отрицательном значении длины
2) Constructor_Copy: корректное копирование битового поля
3) SetBit_GetBit_ClrBit: установка, получение и сброс битов с проверкой граничных случаев
4) GetLength: корректность возвращаемого значения длины
5) Operation_ASSINGMENT: операция присваивания
6) Operation_COMPARISON: проверка на операторы равенства и неравенства битовых полей
7) Operation_OR/AND/NOT: три теста на проверку побитовых операций

Набор проверок TestTSet (9 тестов):

1) Create_Set: создание множества и проверка на исключение при отрицательной длине
2) Constructor_Copy: копирование множеств, создание используя BitField
3) InsElem_IsMember_DelElem: добавление, проверка принадлежности и удаление элементов
4) Operation_COMPARISON: сравнение множеств
5) Operation_ASSINGNMENT: операция присваивания
6) Operation_With_Int: добавление и удаление элемента через операции с целым числом
7) Operation_Union/Intersection: объединение и пересечение множеств одного универса

8) Operation_Complement: операция дополнения множества

Результаты выполнения тестов: На рисунке (Рис. 1) представлен вывод консоли по итогам выполнения программы.

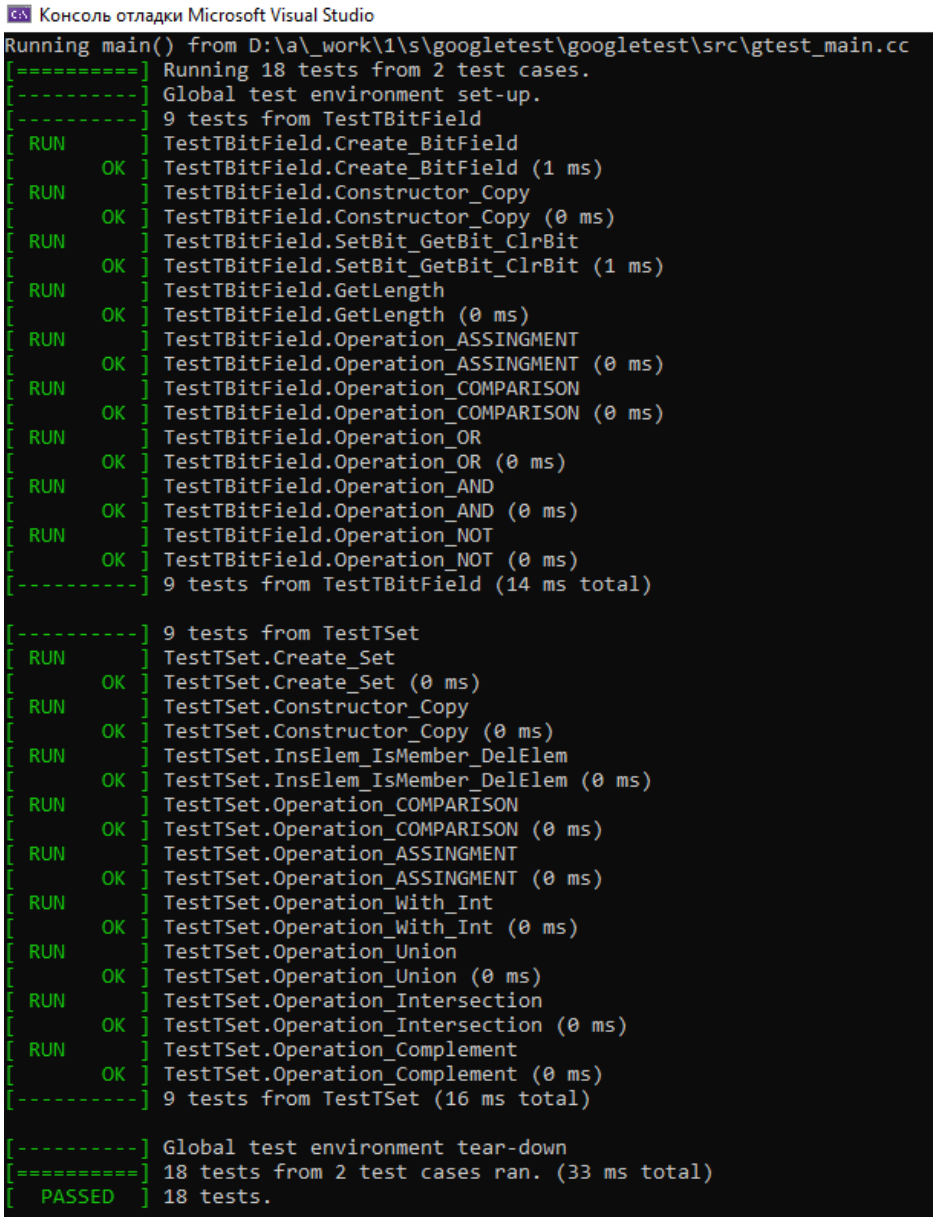


Рис. 1 Результаты работы программы

Все 18 тестов из 2 наборов были запущены успешно, итоговая строка `[ PASSED ] 18 tests` подтверждает, что все тесты пройдены с положительным статусом. Общее время работы составило 33ms.

Заключение

В ходе выполнения лабораторной работы решена задача реализации конечных множеств на основе битовых полей на языке C++. Разработаны два класса:

1. TBitField – битовое поле, реализует динамическое выделение памяти под элементы, корректное копирование, работу с отдельными элементами и полный набор побитовых операций.
2. TSet – использует битовое поле и реализует основные теоретико-множественные операции.

Корректность работы программы подтверждают реализованные тесты, покрывающие как стандартные сценарии, так и граничные случаи. Поставленная задача выполнена в полном объёме.

Литература

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация GTest: https://google.github.io/googletest/